Introducing Workflow Management to Computational Social Science

Andreas Kuepfer
TU Darmstadt

Christian Arnold
University of Birmingham

8th Annual COMPTEXT Conference (April 23-25, 2026) | University of Birmingham, UK

TABLE 1 AI-powered text analysis workflow.

1. Input texts and validation data

a. Text corpus:

i. Party manifestos from the Manifesto Project (MP)

ii. Coalition agreements from Klüver et al. (2023)

b. Positions on six issue dimensions from expert surveys fielded in election years:

i. Laver and Hunt (1992); Benoit and Laver (2006)

ii. Chapel Hill Expert Survey

c. Issue positions on same six dimensions derived from scaling hand-coded labels:

i. MP data for party manifestos

ii. Klüver et al. (2023) data for coalition agreements

2. Large language model (LLM) authorization

Establish high-tier API access to three commercial LLMs: OpenAI's GPT-4o, Anthropic's Claude 3.5 Sonnet, Google's Gemini 1.5 Pro

3. Summarize texts

a. Present original language texts to LLMs for processing

b. Prompt LLMs to generate 200–300-word English language summaries of each text's content relating to each of the six issue dimensions.

4. Scale text summaries

Prompt LLMs to locate positions of each summary on the specified 7-point scales

a. for "zero-shot" learning (no examples)

b. for "few-shot" learning (adding three sample summaries and associated benchmark positions for each scale)

5. Validation

a. *Concurrent validity*: Correlate ensemble means of LLM estimates of party positions with ensemble means of expert judgments from 1b

b. *Reliability*: As a benchmark for reliability, compare LLM-expert correlations with expert–expert correlations.

c. *Predictive validity*: Compare ensemble means of LLM estimates of coalitions' issue positions with analogous hand-coded estimates from 1c, to see which better accords with a prediction common to almost all models of government formation.

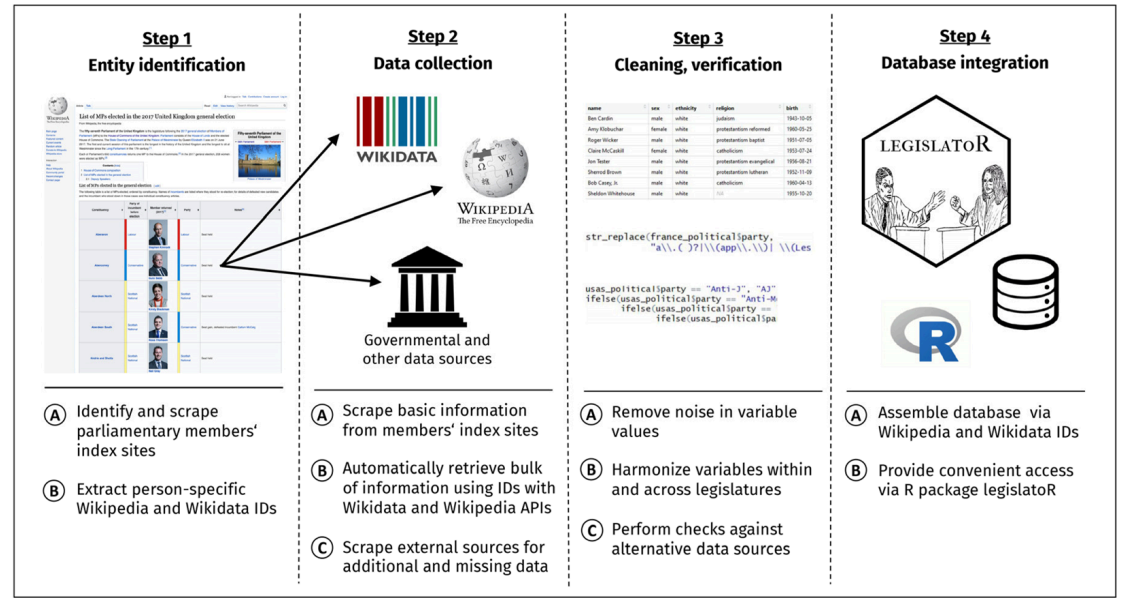
6. Replication and robustness checks

Repeat Steps 2–5a to see if results are stable.

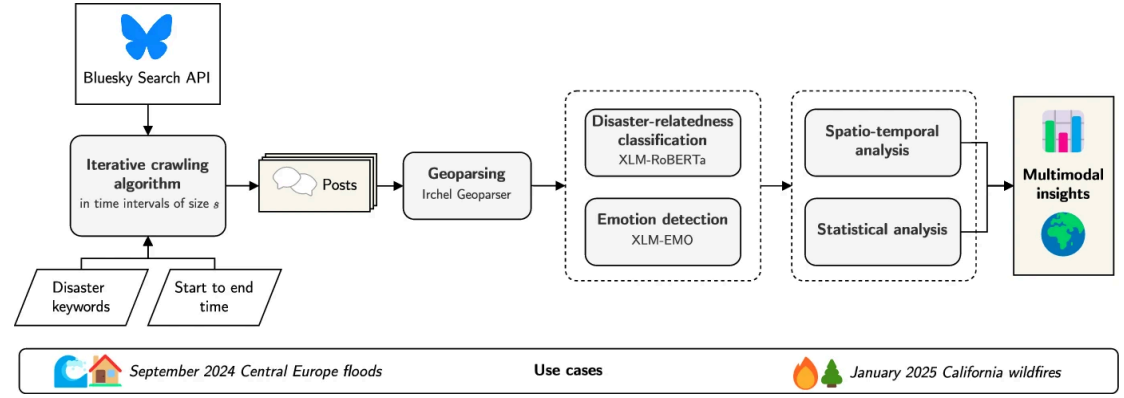
Replicate results using three open-weight LLMs: DeepSeek-V3 (671B), LLaMA-3.3 (70B), and Gemma-3-27B-it.

Notes: This lists each of the steps in the workflow generating the results reported in this paper.

(Benoit et al., 2026)



(Göbel and Munzert, 2018)



(Hanny et al., 2025)

The Replication Crisis is an Infrastructure Crisis

CSS pipelines have grown:

- Large-n corpora: text, audio, image, video
- Multi-step processing chains involving different models and environments
- Workloads exceeding a single machine

But replication infrastructure has not kept pace: mostly still a collection of standalone scripts run by hand.

Six Infrastructure Issues in Current Practice

Automation

Scripts executed by hand, in mental order

→ Stale outputs; execution errors go undetected

Scalability

Code designed to fit a laptop

→ Computationally intensive questions stay unanswered

Portability

Hardcoded paths, undeclared dependencies

→ Code that works on one machine breaks silently on another

Readability

Scripts written for one-time use

→ Replication is nominal; reuse is impossible

Technical Documentation

No automatic logging

→ Results cannot be traced to conditions of production

Analytical Documentation

Decisions undocumented in code

→ Researcher degrees of freedom remain invisible

Workflow Management Systems

A systematic solution to an infrastructure problem

Workflow Management Systems (WMS) Offer a Systematic Solution

A WMS represents a pipeline as an explicit **DAG**: each step declares its **inputs** and **outputs** while the system enforces order, detects stale outputs, and manages environments automatically.

Why Snakemake?

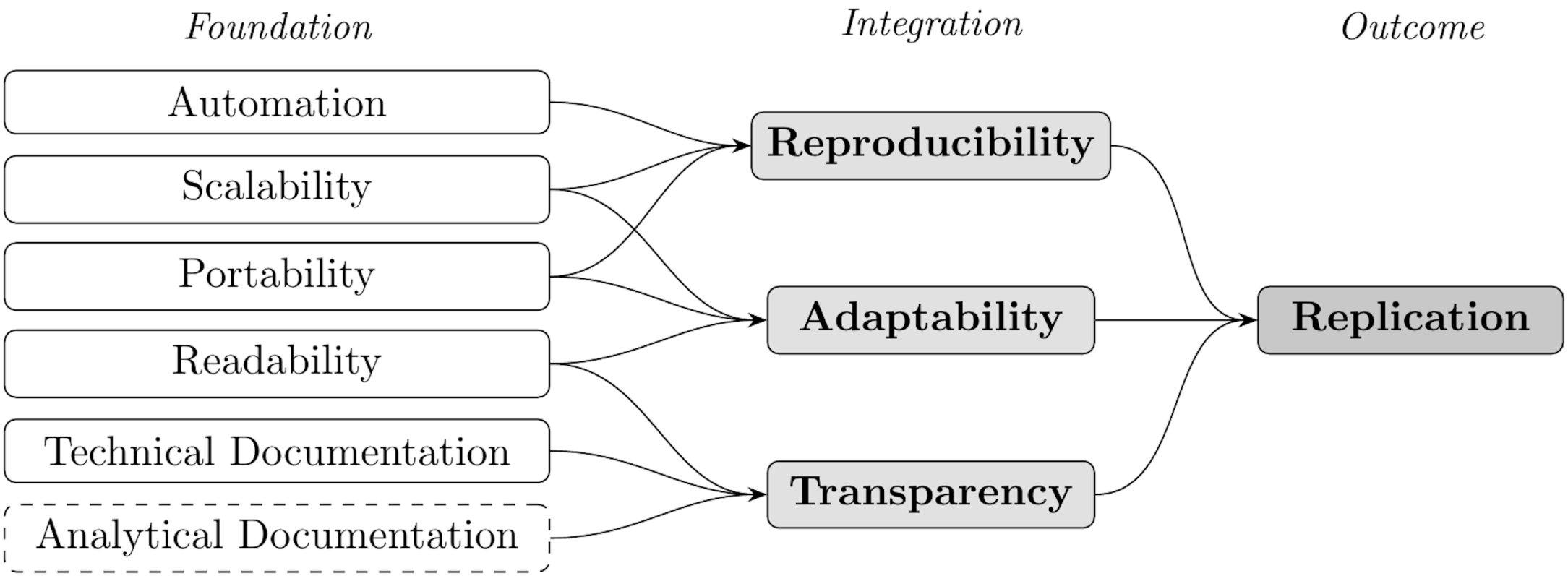
- Python-based DSL (low barrier for CSS researchers)
- Fastest end-to-end execution among WMS benchmarks *(Larsonneur et al., 2018)*
- 75% of public Snakemake repos have less than 23 rules *(Pohl et al., 2024)*
- Hundreds of citations annually in STEM; social sciences account for just >1% *(Mölder et al., 2021)*

```
    ...
        json=SPEECHES / "{parliament}" / "{filename}" / "features" /
          "{base}_{segment}_depth.json",
    script: "scripts/generate_video_feature_depth.py"

rule generate_video_feature_faces:
    input:
        video=SPEECHES / "{parliament}" / "{filename}" / "videos" /
          "{base}_{segment}.mp4",
        depth_json=SPEECHES / "{parliament}" / "{filename}" / "features" /
          "{base}_{segment}_depth.json",
    output:
        json=SPEECHES / "{parliament}" / "{filename}" / "features" /
          "{base}_{segment}_faces.json",
    script: "scripts/generate_video_feature_faces.py"

rule generate_video_feature_headpose:
    input:
        faces_json=SPEECHES / "{parliament}" / "{filename}" / "features" /
          "{base}_{segment}_faces.json",
    ...
```

Framework: From Six Properties to Full Replication (Adapted from *Mölder et al. (2021)*)



How WMS Addresses the Six Issues

Reproducibility | Adaptability | Transparency

Reproducibility: Automation & Portability in Practice

Automation

- Snakemake infers the full DAG from rule definitions
- Execution order is enforced by the software, not the researcher's memory
- A change to any upstream step invalidates stale downstream outputs (automatic rerun)

Portability

- `conda:` directive pins the exact software environment per rule
- `singularity:/apptainer:` enables full container support
- Hardcoded paths replaced by Snakemake's wildcard system

⚠ **The two most common causes of replication failure** ***(Eubank 2016)***

Wrong execution order and environment/version inconsistency

Snakemake resolves both issues systematically. It also automatically satisfies the REP framework *(Stodden et al., 2016)*.

Adaptability: Scale Without Porting

Without accessible scaling infrastructure, studies are designed to fit a laptop. In turn, this removes the incentive to invest in scaling infrastructure.

With Snakemake, the same workflow definition runs on different environments with a single config change:

- A local laptop
- A SLURM/PBS HPC cluster
- Google Cloud
- AWS, Azure

Without WMS	With WMS
Prototype locally, manually port to HPC	Same file, one config
Ad-hoc Virtual Machine deployment	Portable, parallelised, reproducible
Studies limited by laptop RAM	Much more resources due to HPC portability

Transparency: Documentation Without Effort

Technical Documentation (automatic)

- Every rule execution is logged: timestamps, input files, output files, software versions, exit codes
- LLM pipelines: model version, temperature, system prompt, and prompt template are encoded in workflow definition or pinned environment files
- No separate README can fall out of sync with a workflow file *that is the workflow*

Analytical Documentation (structured)

- `config.yaml` provides a machine-readable location for all analytical parameters
- Model specifications, sample restrictions, exclusion criteria, operationalisation choices, robustness variants
- Each parameter = a named, versioned, version-controlled entry

Snakemake in Practice

30,000 hours of parliamentary video across 17 German parliaments

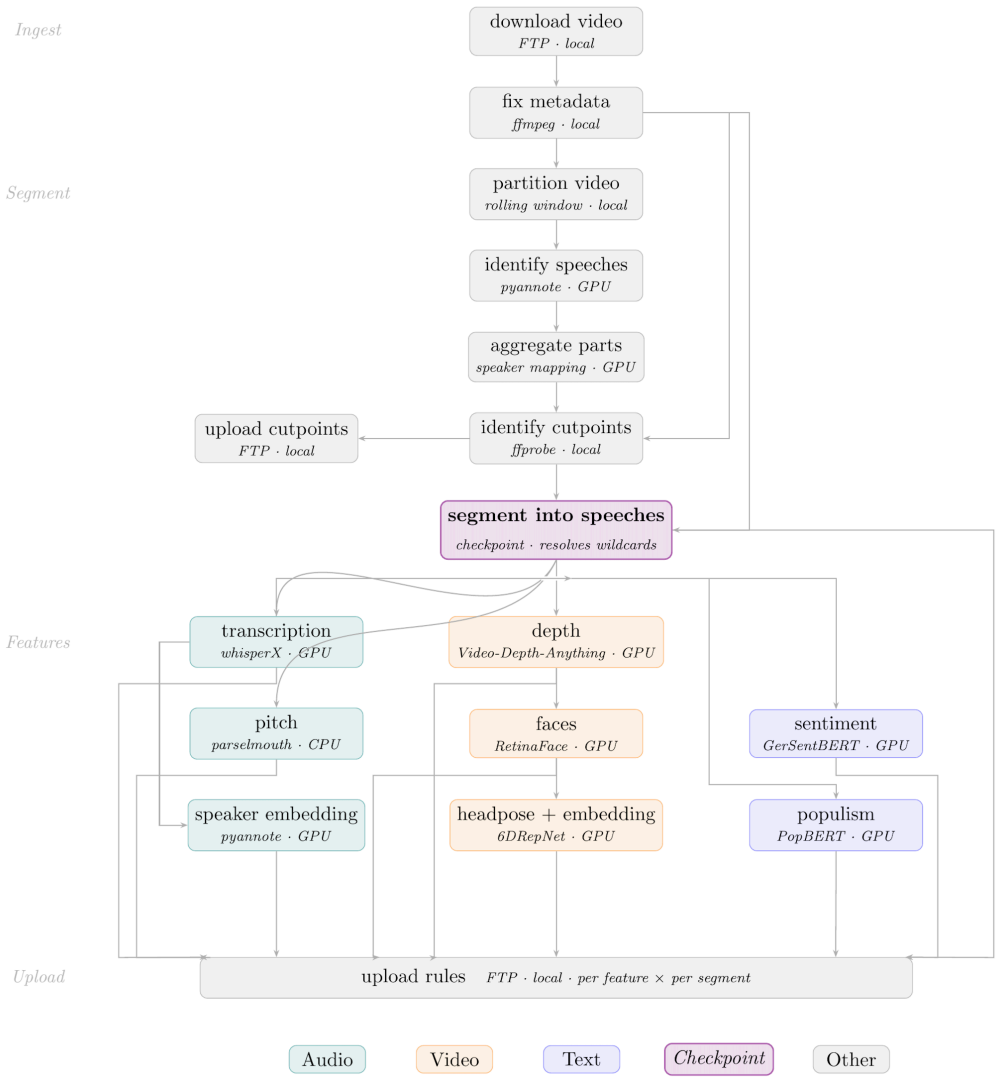
Application: A Snakemake Pipeline for 30,000 Hours of Parliamentary Video

The corpus

- All available speeches from the German Bundestag and all 16 state parliaments
- >30,000 hours (>20TB) of footage
- ~170,000 individual uncut video files
- Heterogeneous across parliaments: format, segmentation, and recording quality vary widely

Performance with Snakemake

~4 min 20 sec end-to-end for 1 hour of raw video



Application: Is Populist Rhetoric Spontaneous or Scripted?

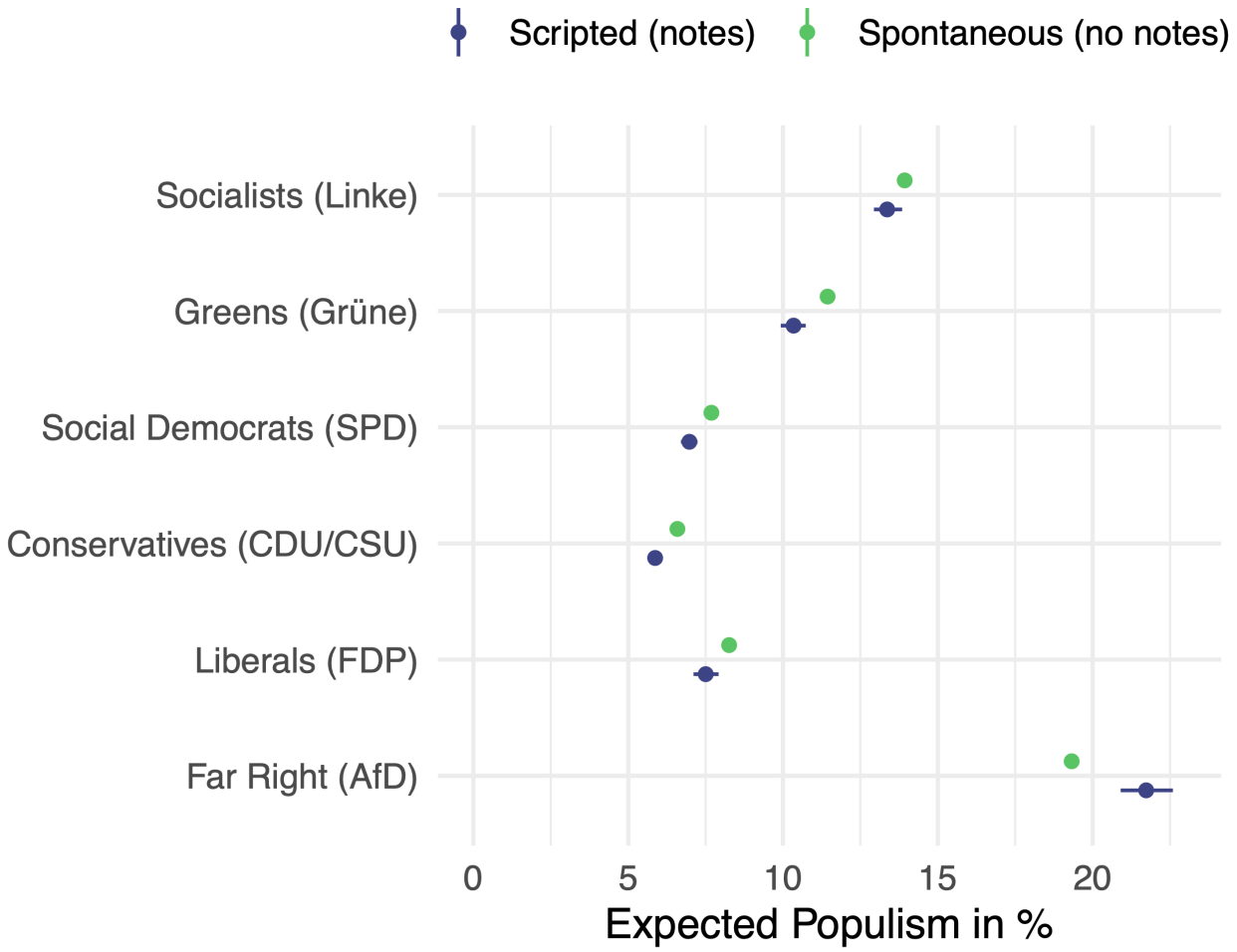
Research question

Does populist rhetoric emerge organically in unscripted speech, or is it prepared in advance?

Gaze direction reveals delivery mode: looking at notes = scripted; looking at audience = more *spontaneous*.

Key findings

- **Mainstream parties**: *spontaneous* → more populist language
- **Far-Right AfD reverses**: scripted → more populist language (populism is a deliberate strategy)



Conclusion

WMS are an implicit implementation of the replication framework the discipline has long dicussed:

- **Automation** removes human error and undocumented execution order
- **Scalability** breaks the infrastructural constraint that keeps CSS artificially smaller
- **Portability** reduces environment inconsistency across machines and over time
- **Readability** makes pipelines comprehensible enough to be reviewed, verified, and built upon
- **Technical Documentation** creates a provenance record automatically
- **Analytical Documentation** makes the garden of forking paths visible and auditable

Together, they fulfil what *King (1995)* demanded: sufficient information for a third party to understand, evaluate, and extend the work.

Introducing Workflow Management to Computational Social Science

Andreas Kuepfer
TU Darmstadt
andreas.kuepfer@tu-darmstadt.de
andreaskuepfer.github.io
ankuepfer.bsky.social

Christian Arnold
University of Birmingham
c.arnold.2@bham.ac.uk
chrisguarnold.github.io
chrisguarnold.bsky.social

8TH ANNUAL COMPTEXT CONFERENCE | APRIL 23–25, 2026 | UNIVERSITY OF BIRMINGHAM, UK